P3477R2
There are exactly 8 bits in a byte

Published Proposal, 2025-01-12

This version:
http://wg21.link/P3477r2

Author:
JF Bastien (Woven by Toyota)

Audience:
CWG, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
github.com/jfbastien/papers/blob/master/source/P3477r2.bs

Table of Contents

1 Revision History

1.1 r0

1.2 r1

1.3 r2

2 Rationale

3 Motivation

4 Impact on C

5 Wording

5.1 Language

5.2 Library

References

Informative References

Abstract: `8`ers gonna `8`.

1. Revision History

1.1. r0

[P3477R0] was the first published version of the paper, prompted by internet denizens nerd-sniping the author into witing the paper. There was much rejoicing

1.2. r1

[P3477r1] was revised a month later, after the internet denizens read the paper and provided substantial feedback regarding exotic architectures, and pointing out some embarrassing typos. In that period, a few internet denizens showed once more that they don't read papers and only read the title, commenting on things that are already in the paper. The author would scold them here, but realizes the futility of even mentioning this shortcoming.

The C++ committee's Evolution Working Group also reviewed the paper, with the following outcome:

> Poll: D3477r1: There are exactly 8 bits in a byte: forward to CWG/LEWG for inclusion in C++26, removing the intptr/uintptr changes.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 9 | 17 | 3 | 4 | 0 |
>
> Result: consensus in favor

1.3. r2

The previous revision was seen by the C++ committee's SG22 C/C++ Liaison Group, with the following outcome:

> We think WG14 might be interested too, perhaps with a change to hosted environments only. No concerns raised from SG22 perspective.

No other changes to the paper besides removing SG22 from the audience list.

§ 2. Rationale

C has the `CHAR_BIT` macro which contains the implementation-defined number of bits in a byte, without restrictions on the value of this number. C++ imports this macro as-is. Many other macros and character traits have values derived from `CHAR_BIT`. While this was historically relevant in computing's early days, modern hardware has overwhelmingly converged on the assumption that a byte is 8 bits. This document proposes that C++ formally mandates that a byte is 8 bits.

Mainstream compilers already support this reality:

- GCC sets a default value of `8` but no upstream target changes the default;
- LLVM sets `__CHAR_BIT__` to `8`; and
- MSVC defines `CHAR_BIT` to `8`.

We can find vestigial support, for example GCC dropped dsp16xx in 2004, and 1750a in 2002. Search the web for more evidence finds a few GCC out-of-tree ports which do not seem relevant to modern C++.

[POSIX] has mandated this reality since POSIX.1-2001 (or IEEE Std 1003.1-2001), saying:

> As a consequence of adding `int8_t`, the following are true:
>
> - A byte is exactly 8 bits.
> - `CHAR_BIT` has the value 8, `SCHAR_MAX` has the value 127, `SCHAR_MIN` has the value -128, and `UCHAR_MAX` has the value 255.
>
> Since the POSIX.1 standard explicitly requires 8-bit char with two's complement arithmetic, it is easier for application writers if the same two's complement guarantees are extended to all of the other standard integer types. Furthermore, in programming environments with a 32-bit long, some POSIX.1 interfaces, such as `mrand48()`, cannot be implemented if long does not use a two's complement representation.

To add onto the reality that POSIX chose in 2001, C++20 has only supported two's complement storage since [P0907r4], and C23 has followed suit.

The overwhelming support for 8-bit bytes in hardware and software platforms means that software written for non-8-bit bytes is incompatible with software written for 8-bit bytes, and vice versa. C and

C++ code targeting non-8-bit bytes are incompatible dialects of C and C++.

Wikipedia quotes the following operating systems as being currently POSIX compliant (and therefore supporting 8-bit bytes):

- AIX
- HP-UX
- INTEGRITY
- macOS
- OpenServer
- UnixWare
- VxWorks
- z/OS

And many others as being formerly compliant, or mostly compliant.

Even StackOverflow, the pre-AI repository of our best knowledge (after Wikipedia), gushes with enthusiasm about non-8-bit byte architectures, and asks which exotic architecture the committee cares about.

This paper cannot succeed without mentioning the PDP-10 (though noting that PDP-11 has 8-bit bytes), and the fact that some DSPs have 16-bit, 24-bit, or 32-bit words treated as "bytes." These architectures made sense in their era, where word sizes varied and the notion of a byte wasn't standardized. Today, nearly every general-purpose and embedded system adheres to the 8-bit byte model. The question isn't whether there are still architectures where bytes aren't 8-bits (there are!) but whether these care about modern C++... and whether modern C++ cares about them.

The example which seems the most relevant of current architecture with non-8-bit-bytes is TI's TMS320C28x, whose compiler manual states:

> The TI compiler accepts C and C++ code conforming to the International Organization for Standardization (ISO) standards for these languages. The compiler supports the 1989, 1999, and 2011 versions of the C language and the 2003 version of the C++ language.

TI has expressed that it does not intend to support C++11 or later. They offer a migration guide from 8-bit bytes.

Another recent DSP which is sometimes brought up is CEVA-TeakLite. The latest generation, CEVA-TeakLite-4, only supports a C compiler.

Yet another potentially relevant architecture is SHARC, whose latest compiler manual explains which architectures support different architectural features in section "Processor Features", and whether the compiler option `-char-size[-8|-32]` is available. In any case, only C++03 and C++11 are supported, with significant features missing (but interesting anachronisms can be enabled, I recommend reading that section of the manual!).

A popular DSP which supports 8-bit bytes is Tensilica, whose compiler is based on clang.

Qualcomm provides three compilers that target their Kalimba series of DSPs. The `kcc`, `kalcc`, and `kalcc32` compilers all appear to be C compilers with no support for C++. These compilers support a 24 bit byte size according to Coverity's configuration, no public documentation seems available to confirm this.

An EDG representative has privately stated:

> I checked the configuration files that our customers share with us. One customer shared a configuration file that sets bytes to 32 as recently as 2022. This would configure a front end that supports C++17 and some C++20.

The author would happily retract this papers or change the proposal if hardware implementors expressed a desire to support modern C++ on their non-8-bit-per-byte hardware.

Does this proposal prevent new weird architectures from being created? Not really! These hypothetical new architectures would write their entire software stack from scratch with or without this paper, and would benefit from C23's `_BitInt` as standardized by [N2763] rather than have `char` and other types of implicit size.

§ 3. Motivation

Why bother? A few reasons:

- The complexity of supporting non-8-bit byte architectures sprinkles small but unnecessary burden in quite a few parts of language and library;
- Compilers and toolchains are required to support edge cases that do not reflect modern usage;
- New programmers are easily confused and find C++'s exotic tastes obtuse;
- Some seasoned programmers joyfully spend time supporting non-existant platforms "for portability" if they are unwise, even writing FAQs about this which others then read and preach as gospel;

- Our language looks silly, solving problems that nobody has.

One reason not to bother: there still are processors with non-8-bit bytes. The question facing us is: are they relevant to modern C++? If we keep supporting the current approach where Bytes"R"Us, will developers who use these processors use the new versions of C++?

A cut-the-baby-in-half alternative is to mandate that `CHAR_BIT % 8 == 0`. Is that even making anything better? Only if the Committee decides to keep supporting Digital Signal Processors (DSPs) and other processors where `CHAR_BIT` is not `8` but is a multiple of `8`.

Another way to cut-the-baby-in-half is to mandate that `CHAR_BIT` be `8` on hosted implementations, and leave implementation freedom on freestanding implementations.

4. Impact on C

This proposal explores whether C++ is relevant to architectures where bytes are not 8 bits, and whether these architectures are relevant to C++. The C committee might reach a different conclusion with respect to this language. Ideally, both committees would be aligned. This papers therefore defers to WG14 and the SG22 liaison group to inform WG21.

5. Wording

5.1. Language

Edit [**intro.memory**] as follows:

> The fundamental storage unit in the ++ memory model is the byte. A byte is 8 bits, which is at least large enough to contain the ordinary literal encoding of any element of the basic character set literal character set and the eight-bit code units of the Unicode UTF-8 encoding form and is composed of a contiguous sequence of bits, the number of which is bits in a byte. The least significant bit is called the low-order bit; the most significant bit is called the high-order bit. The memory available to a C++ program consists of one or more sequences of contiguous bytes. Every byte has a unique address.
>
> > The number of bits in a byte is reported by the macro `CHAR_BIT` in the header `climits`. Its value is 8.

§ 5.2. Library

Edit [**climits.syn**] as follows:

```
// all freestanding
#define CHAR_BIT see below8
#define SCHAR_MIN see below-128
#define SCHAR_MAX see below127
#define UCHAR_MAX see below255
#define CHAR_MIN see below
#define CHAR_MAX see below
#define MB_LEN_MAX see below
#define SHRT_MIN see below
#define SHRT_MAX see below
#define USHRT_MAX see below
#define INT_MIN see below
#define INT_MAX see below
#define UINT_MAX see below
#define LONG_MIN see below
#define LONG_MAX see below
#define ULONG_MAX see below
#define LLONG_MIN see below
#define LLONG_MAX see below
#define ULLONG_MAX see below
```

The header `climits` defines all macros the same as the C standard library header `limits.h` , except as noted above .

> Except for `CHAR_BIT` and `MB_LEN_MAX`, a macro referring to an integer type `T` defines a constant whose type is the promoted type of `T`.

Edit [**cstdint.syn**] as follows:

The header `cstdint` supplies integer types having specified widths, and macros that specify limits of integer types.

```
int8_t
int16_t
int32_t
int64_t
int_fast8_t
int_fast16_t
int_fast32_t
int_fast64_t
int_least8_t
int_least16_t
int_least32_t
int_least64_t
intmax_t
intptr_t
uint8_t
uint16_t
uint32_t
uint64_t
uint_fast8_t
uint_fast16_t
uint_fast32_t
uint_fast64_t
uint_least8_t
uint_least16_t
uint_least32_t
uint_least64_t
uintmax_t
uintptr_t

// all freestanding
namespace std {
  using int8_t         = signed integer type;   // optional
  using int16_t        = signed integer type;   // optional
  using int32_t        = signed integer type;   // optional
  using int64_t        = signed integer type;   // optional
  using intN_t         = see below;             // optional

  using int_fast8_t    = signed integer type;
  using int_fast16_t   = signed integer type;
  using int_fast32_t   = signed integer type;
  using int_fast64_t   = signed integer type;
```

```
  using int_fastN_t    = see below;             // optional

  using int_least8_t   = signed integer type;
  using int_least16_t  = signed integer type;
  using int_least32_t  = signed integer type;
  using int_least64_t  = signed integer type;
  using int_leastN_t   = see below;             // optional

  using intmax_t       = signed integer type;
  using intptr_t       = signed integer type;   // optional

  using uint8_t        = unsigned integer type; // optional
  using uint16_t       = unsigned integer type; // optional
  using uint32_t       = unsigned integer type; // optional
  using uint64_t       = unsigned integer type; // optional
  using uintN_t        = see below;             // optional

  using uint_fast8_t   = unsigned integer type;
  using uint_fast16_t  = unsigned integer type;
  using uint_fast32_t  = unsigned integer type;
  using uint_fast64_t  = unsigned integer type;
  using uint_fastN_t   = see below;             // optional

  using uint_least8_t  = unsigned integer type;
  using uint_least16_t = unsigned integer type;
  using uint_least32_t = unsigned integer type;
  using uint_least64_t = unsigned integer type;
  using uint_leastN_t  = see below;             // optional

  using uintmax_t      = unsigned integer type;
  using uintptr_t      = unsigned integer type; // optional
}

#define INTN_MIN         see below
#define INTN_MAX         see below
#define UINTN_MAX        see below

#define INT_FASTN_MIN    see below
#define INT_FASTN_MAX    see below
#define UINT_FASTN_MAX   see below

#define INT_LEASTN_MIN   see below
#define INT_LEASTN_MAX   see below
```

```
#define UINT_LEASTN_MAX  see below

#define INTMAX_MIN       see below
#define INTMAX_MAX       see below
#define UINTMAX_MAX      see below

#define INTPTR_MIN       see below              // optional
#define INTPTR_MAX       see below              // optional
#define UINTPTR_MAX      see below              // optional

#define PTRDIFF_MIN      see below
#define PTRDIFF_MAX      see below
#define SIZE_MAX         see below

#define SIG_ATOMIC_MIN   see below
#define SIG_ATOMIC_MAX   see below

#define WCHAR_MIN        see below
#define WCHAR_MAX        see below

#define WINT_MIN         see below
#define WINT_MAX         see below

#define INTN_C(value)    see below
#define UINTN_C(value)   see below
#define INTMAX_C(value)  see below
#define UINTMAX_C(value) see below
```

The header defines all types and macros the same as the C standard library header `stdint.h` , except that none of the types nor macros for 8, 16, 32, nor 64 are optional because bytes are 8 bits .

All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, or 64. ~~The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, and 64 are also optional; however, i~~ I f an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding typedef-names. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding typedef-name.

> The macros `INTN_C` and `UINTN_C` correspond to the typedef-names `int_leastN_t` and `uint_leastN_t`, respectively.

Within [**localization**], remove the 4 *mandates* clauses specifying:

> ~~`CHAR_BIT == 8` is `true`~~

Within [**cinttypes.syn**], do not change the list of `PRI` macros, and leave this paragraph as-is:

> Each of the `PRI` macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in [**cstdint.syn**]. Each of the `SCN` macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in [[**cstdint.syn**]] and has a suitable `fscanf` length modifier for the type.

§ References

§ Informative References

[N2763]
Aaron Ballman; et al. *Adding a Fundamental Type for N-bit integers*. 2021-06-21. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2763.pdf

[P0907r4]
JF Bastien. *Signed Integers are Two's Complement*. 6 October 2018. URL: https://wg21.link/p0907r4

[P3477R0]
JF Bastien. *There are exactly 8 bits in a byte*. 16 October 2024. URL: https://wg21.link/p3477r0

[P3477r1]
JF Bastien. *There are exactly 8 bits in a byte*. 21 November 2024. URL: https://wg21.link/p3477r1

[POSIX]
The Open Group Base Specifications Issue 8. 2024. URL: https://pubs.opengroup.org/onlinepubs/9799919799/